

Requirement Analysis Phase

Technology Stack(Architecture & Stack)

Date	October 3, 2026
Team ID	SWTID-2026-7210
Project Name	Script Controlled ACL: Restrict Record Access Based On Field Value
Maximum Marks	4 Marks

1. Technical Architecture Description

The core system uses a high-performance database layer combined with an execution hook engine. When a user attempts an execution step to view data, a database interceptor extracts the contextual fields (such as security status or record sensitivity flags). This evaluation request is handed off to a sandboxed runtime script evaluation sandbox. If authorization passes, structural objects are returned; if denied, records are dynamically filtered or data masking layers hide sensitive information. Comprehensive cryptographic audit logs track the lifecycle of each evaluation request to maintain dynamic compliance proofs.

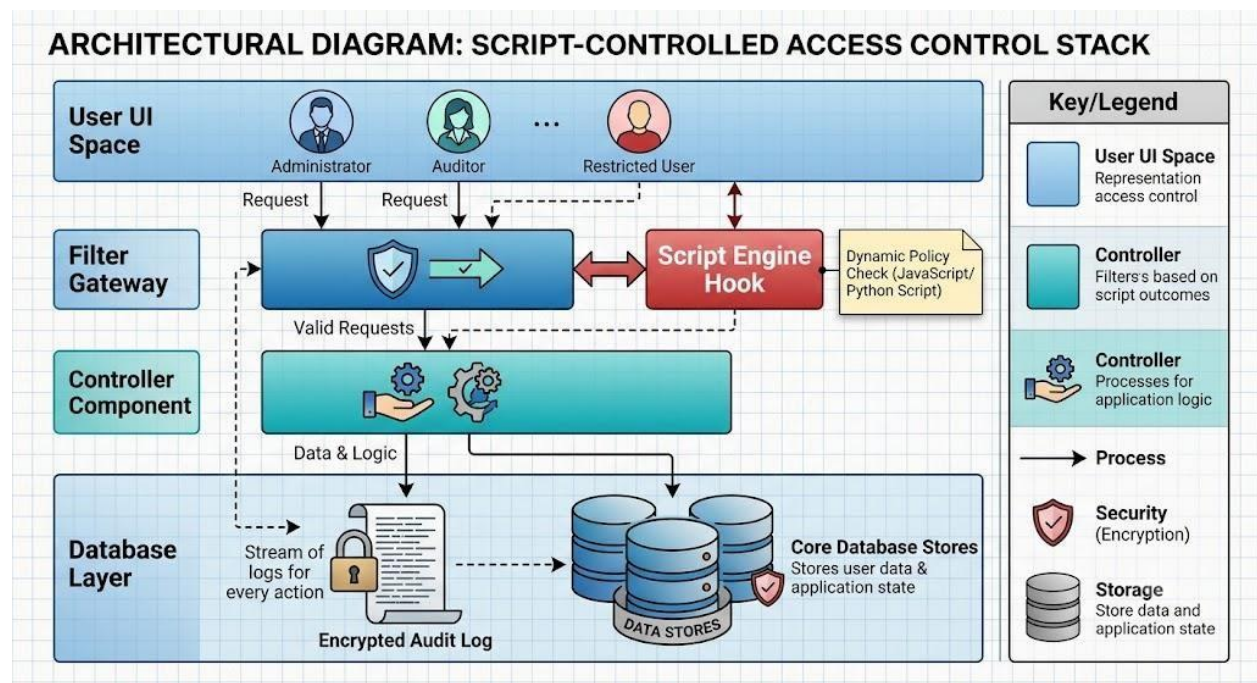


Table-1 : Components & Technologies

S.No	Component	Description	Technology
------	-----------	-------------	------------

1	User Interface	Web management layout workspace dashboard used by network security team leaders to write policies and configure conditional	HTML5, CSS3, Modern JavaScript (ES6+), React.js, Tailwind CSS Framework elements.
----------	----------------	---	---

		runtime script templates.	
2	Application Logic-1	Central processing module hosting API interceptors, dynamic contextual parameter parser systems, and schema processing utilities.	Python v3.10+ / Node.js Runtime Platform engine configurations.
3	Application Logic-2	Isolated sandboxed runtime scripting evaluation core designed to parse operational syntax trees without risking full system loop exceptions.	JavaScript Engine Context (vm2 Sandbox Core) or Python Restricted Execution environment wrapper hooks.
4	Application Logic-3	Automated real-time background context engine analyzing database query inputs, mapping them into user sessions dynamically.	Custom interceptor middleware hooks running on Express framework modules.
5	Database	Structured relation layer preserving master files, dynamic user metadata, database role properties, and organizational record models.	MySQL Database Engine or PostgreSQL structural schema containers.
6	Cloud Database	Cloud system database instance syncing live schema details across regions to handle distributed highvolume corporate policy matrices.	IBM DB2 on Cloud or AWS RDS Cluster instances running multi-availability data synchronization structures.

7	File Storage	High-integrity physical logs archive repository where backup historical audit dumps are preserved for data compliance reviews.	IBM Block Storage, AWS S3 Buckets, or localized protected Linux Filesystems.
8	Infrastructure (Server / Cloud)	Cloud infrastructure configuration layout supporting deployment frameworks and microservices application architecture layers.	Local Development environment configs / Cloud Kubernetes Container Engine clusters.

Table-2: Application Characteristics

S.No	Characteristics	Description	Technology Used
1	Open-Source Frameworks	Utilization of robust verified open-source framework engines to avoid proprietary software vendor dependency issues.	React JS interface systems, Python Frameworks, Node libraries, and ObjectRelational Mapping (ORM) tools.
2	Security Implementations	Strict end-to-end security validations, token verification pipelines, and cryptographic query validation checks.	SHA-256 Hashing algorithms, JSON Web Tokens (JWT) context structures, and OWASP standard sanitization layers.
3	Scalable Architecture	System capacity structure to scale dynamically whenever thousands of parallel script parameters trigger simultaneously.	Microservices Application Setup combined with Docker orchestration containers and load balancers.
4	Availability	High-uptime layout strategies ensuring execution engines resolve conditional paths instantly without single-point service dropdowns.	Distributed cluster networks, redundant backup instances, and multi-region continuous live failovers.

5	Performance	Optimization metrics engineered to minimize code script parsing overhead delays during fast analytical database searches.	Redis Logic Caching layers, Data Indexing properties, and Content Delivery Networks (CDNs).
---	-------------	---	---